

```
GOOGLE_MAPS_API_KEY = "AIzaSyCOEo65FvdvFDC0chQq9AWDdyJehssMxdY"
```

```
print("API key loaded")
```

```
API key loaded
```

```
# =====
# CELL 2 - GOOGLE SATELLITE MAP + 4 CORNER SELECTION
# =====

from google.colab import output
from IPython.display import display, HTML
import json

# -----
# DEFAULT LOCATION
# Change these if you want another location
# -----

LAT = 22.7196      # Indore
LON = 75.8577
ZOOM = 19

# -----
# GOOGLE MAP
# -----

html = f"""
<!DOCTYPE html>
<html>
<head>

<style>

html, body {{
    margin: 0;
    padding: 0;
    width: 100%;
    height: 100%;
}}

#map {{
    width: 100%;
    height: 650px;
}}

#panel {{
    position: absolute;
    top: 15px;
    left: 15px;
    z-index: 999;

    background: white;
    padding: 15px;

    border-radius: 10px;

    font-family: Arial;

    box-shadow: 0 2px 10px rgba(0,0,0,0.3);
}}

button {{
    margin-top: 8px;
    padding: 10px 15px;
    border: none;
    border-radius: 6px;
    cursor: pointer;
}}

#submit {{
```

```

        background: #16a34a;
        color: white;
    }}

    #reset {{
        background: #dc2626;
        color: white;
    }}

</style>

</head>

<body>

<div id="panel">

<b>Parking Space Optimizer</b>

<br><br>

Click the <b>4 corners</b> of the parking area.

<br>

Selected:
<span id="count">0</span> / 4

<br>

<button id="submit" onclick="submitPoints()">
SUBMIT AREA
</button>

<button id="reset" onclick="resetPoints()">
RESET
</button>

</div>

<div id="map"></div>

<script>

let map;

let points = [];

let markers = [];

let polygon = null;

// -----
// INITIALIZE MAP
// -----

function initMap() {{

    const center = {{
        lat: {LAT},
        lng: {LON}
    }};

    map = new google.maps.Map(
        document.getElementById("map"),
        {{
            center: center,

            zoom: {ZOOM},

```

```
        mapTypeId: "satellite",

        tilt: 0,

        streetViewControl: false,

        fullscreenControl: true,

        mapTypeControl: true

    }}
);

// -----
// CLICK EVENT
// -----

map.addListener("click", function(event) {{

    if(points.length >= 4)
        return;

    const point = {{

        lat: event.latLng.lat(),

        lng: event.latLng.lng()

    }};

    points.push(point);

    // Marker

    const marker = new google.maps.Marker({{

        position: point,

        map: map,

        label: String(points.length)

    }});

    markers.push(marker);

    document.getElementById("count").innerText =
        points.length;

    // Draw polygon after 4 points

    if(points.length === 4) {{

        polygon = new google.maps.Polygon({{

            paths: points,

            strokeColor: "#00ff00",

            strokeOpacity: 1,

            strokeWeight: 3,

            fillColor: "#00ff00",

            fillOpacity: 0.20,
```

```

        map: map

    });

}

});

// -----
// RESET
// -----

function resetPoints() {{

    points = [];

    markers.forEach(function(marker) {{

        marker.setMap(null);

    }});

    markers = [];

    if(polygon != null) {{

        polygon.setMap(null);

        polygon = null;

    }}

    document.getElementById("count").innerText = "0";

}}

// -----
// SUBMIT
// -----

function submitPoints() {{

    if(points.length != 4) {{

        alert(
            "Please select exactly 4 corners."
        );

        return;

    }}

    // Send data back to Colab

    google.colab.kernel.invokeFunction(
        "parking.receive_points",
        [points],
        {}
    );

}}

</script>

```

```

<script
  async
  src="https://maps.googleapis.com/maps/api/js?key={GOOGLE_MAPS_API_KEY}&loading=async&callback=initMap">
</script>

</body>
</html>
"""

# -----
# CALLBACK
# -----

received_points = None

def receive_points(points):

    global received_points

    received_points = points

    print("\n=====")
    print("PARKING AREA SELECTED")
    print("=====")

    for i, p in enumerate(points):

        print(
            f"P{i+1}: "
            f"lat={p['lat']:.8f}, "
            f"lng={p['lng']:.8f}"
        )

    output.register_callback(
        "parking.receive_points",
        receive_points
    )

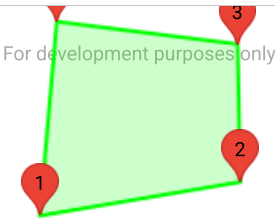
display(HTML(html))

```

Parking Space Optimizer

Click the 4 corners of the parking area.
Selected: 4 / 4

SUBMIT AREA RESET



Imagery ©2026 Airbus, CNES / Airbus, Maxar Technologies Report a map error

```
=====
PARKING AREA SELECTED
=====
P1: lat=22.71985525, lng=75.85800092
P2: lat=22.71991463, lng=75.85838448
P3: lat=22.72015709, lng=75.85837911
P4: lat=22.72019667, lng=75.85803311
```

```
# =====
# CELL 3 – ACCURATE GEO → METRIC PARKING GEOMETRY
# =====

!pip -q install pyproj shapely

import math
import numpy as np

from pyproj import CRS, Transformer
from shapely.geometry import Polygon

# =====
# CHECK INPUT
# =====

if received_points is None:
    raise RuntimeError(
        "Run Cell 2 and submit exactly 4 points first."
    )

# =====
# WGS84 POINTS
# =====

lats = np.array([
    p["lat"] for p in received_points
])
```

```

lons = np.array([
    p["lng"] for p in received_points
])

# =====
# AUTOMATIC UTM ZONE
# =====

mean_lon = float(np.mean(lons))
mean_lat = float(np.mean(lats))

utm_zone = int(
    math.floor((mean_lon + 180) / 6) + 1
)

if mean_lat >= 0:

    epsg = 32600 + utm_zone

else:

    epsg = 32700 + utm_zone

print("=====")
print("COORDINATE SYSTEM")
print("=====")

print(
    f"Mean latitude : {mean_lat:.8f}"
)

print(
    f"Mean longitude : {mean_lon:.8f}"
)

print(
    f"UTM zone      : {utm_zone}"
)

print(
    f"EPSG          : {epsg}"
)

# =====
# TRANSFORM
# =====

transformer = Transformer.from_crs(
    "EPSG:4326",
    f"EPSG:{epsg}",
    always_xy=True
)

utm_points = []

for lon, lat in zip(lons, lats):

    x, y = transformer.transform(
        lon,
        lat
    )

    utm_points.append(
        (x, y)
    )

# =====

```

```

# PARKING POLYGON
# =====

PARKING_POLYGON_UTM = Polygon(
    utm_points
)

# Fix minor geometry problems if any
if not PARKING_POLYGON_UTM.is_valid:

    PARKING_POLYGON_UTM = (
        PARKING_POLYGON_UTM
        .buffer(0)
    )

# =====
# BASIC METRICS
# =====

PARKING_AREA_M2 = (
    PARKING_POLYGON_UTM.area
)

PARKING_PERIMETER_M = (
    PARKING_POLYGON_UTM.length
)

print("\n=====")
print("PARKING GEOMETRY")
print("=====")

print(
    f"Area      : {PARKING_AREA_M2:.2f} m²"
)

print(
    f"Perimeter : {PARKING_PERIMETER_M:.2f} m"
)

# =====
# SAVE
# =====

PARKING_EPSG = epsg

print("\n✓ Accurate metric geometry ready.")

```

```

=====
COORDINATE SYSTEM
=====
Mean latitude : 22.72003091
Mean longitude : 75.85819940
UTM zone      : 43
EPSG          : 32643

=====
PARKING GEOMETRY
=====
Area      : 1209.30 m²
Perimeter : 140.53 m

✓ Accurate metric geometry ready.

```

```

# =====
# CELL 4 – RESEARCH-BACKED PARKING OPTIMIZER
# =====

import numpy as np
import math
import matplotlib.pyplot as plt

```

```

from shapely.geometry import box
from shapely.affinity import rotate, translate

# =====
# PARKING PROFILE
# =====

# BIS SP 73:2023 shared car parking:
# 2.75 m x 5.00 m

SLOT_WIDTH = 2.75
SLOT_LENGTH = 5.00

# =====
# AISLE PROFILE
# =====

# Engineering defaults for the prototype.
#
# 45° / 60°:
# one-way circulation
#
# 90°:
# two-way circulation
#
# These are configurable engineering assumptions,
# NOT a substitute for local authority approval.

AISLE_WIDTHS = {
    0: 3.60,
    45: 4.80,
    60: 4.80,
    90: 6.00
}

def get_aisle_width(angle):
    """
    Interpolate between standard anchor values.
    """

    known_angles = np.array([
        0,
        45,
        60,
        90
    ])

    known_widths = np.array([
        3.60,
        4.80,
        4.80,
        6.00
    ])

    return float(
        np.interp(
            angle,
            known_angles,
            known_widths
        )
    )

# =====
# ANGLE SEARCH

```

```

# =====
ANGLE_STEP = 1.0

ANGLES = np.arange(
    0,
    90.01,
    ANGLE_STEP
)

# =====
# GEOMETRY HELPERS
# =====

def slot_geometry(
    x,
    y,
    angle
):
    """
    Creates a parking slot centered at x,y.

    Parking angle:
    90° = perpendicular
    45° = diagonal
    0° = parallel
    """

    slot = box(
        -SLOT_WIDTH / 2,
        -SLOT_LENGTH / 2,

        SLOT_WIDTH / 2,
        SLOT_LENGTH / 2
    )

    # Convert parking angle to rectangle rotation.
    rotation = 90 - angle

    slot = rotate(
        slot,
        rotation,
        origin="center",
        use_radians=False
    )

    slot = translate(
        slot,
        xoff=x,
        yoff=y
    )

    return slot

# =====
# PLOT ORIENTATION
# =====

def longest_edge_orientation(
    polygon
):
    coords = list(
        polygon.exterior.coords
    )

    best_length = -1
    best_angle = 0

    for i in range(
        len(coords) - 1

```

```

    ):

        x1, y1 = coords[i]

        x2, y2 = coords[i + 1]

        dx = x2 - x1
        dy = y2 - y1

        length = math.hypot(
            dx,
            dy
        )

        angle = math.degrees(
            math.atan2(
                dy,
                dx
            )
        )

        if length > best_length:

            best_length = length
            best_angle = angle

    return best_angle

# =====
# ROTATE PLOT INTO LOCAL COORDINATES
# =====

DOMINANT_ANGLE = (
    longest_edge_orientation(
        PARKING_POLYGON_UTM
    )
)

print("=====")
print("SITE ORIENTATION")
print("=====")

print(
    f"Dominant edge angle = "
    f"{DOMINANT_ANGLE:.2f}°"
)

LOCAL_POLYGON = rotate(
    PARKING_POLYGON_UTM,
    -DOMINANT_ANGLE,
    origin="centroid",
    use_radians=False
)

# =====
# MODULE DIMENSIONS
# =====

def module_dimensions(
    angle
):

    theta = math.radians(
        angle
    )

    # Perpendicular depth of one stall
    stall_depth = (
        SLOT_LENGTH * math.sin(theta)
    +

```

```

        SLOT_WIDTH * math.cos(theta)
    )

    # Length consumed along aisle
    stall_frontage = (
        SLOT_LENGTH * math.cos(theta)
        +
        SLOT_WIDTH * math.sin(theta)
    )

    aisle = get_aisle_width(
        angle
    )

    # Double-sided parking module
    module_depth = (
        stall_depth
        +
        aisle
        +
        stall_depth
    )

    return (
        stall_depth,
        stall_frontage,
        aisle,
        module_depth
    )

# =====
# BUILD ONE BAY
# =====

def build_bay(
    polygon,
    base_y,
    angle
):

    (
        stall_depth,
        frontage,
        aisle_width,
        module_depth
    ) = module_dimensions(
        angle
    )

    minx, miny, maxx, maxy = (
        polygon.bounds
    )

    # -----
    # AISLE
    # -----

    aisle = box(
        minx,
        base_y + stall_depth,
        maxx,
        base_y + stall_depth + aisle_width
    )

    # Require reasonable aisle coverage
    aisle_intersection = (
        aisle.intersection(
            polygon
        )
    )

```

```

aisle_ratio = (
    aisle_intersection.area
    /
    max(
        aisle.area,
        1e-9
    )
)

# -----
# PARKING ROWS
# -----

row1_y = (
    base_y
    +
    stall_depth / 2
)

row2_y = (
    base_y
    +
    stall_depth
    +
    aisle_width
    +
    stall_depth / 2
)

row1 = []
row2 = []

# -----
# X POSITIONS
# -----

# Slight phase shifts can improve irregular-edge fit.
x_start = (
    minx
    +
    frontage / 2
)

x_positions = np.arange(
    x_start,
    maxx,
    frontage
)

# -----
# FIRST ROW
# -----

for x in x_positions:

    slot = slot_geometry(
        x,
        row1_y,
        angle
    )

    if polygon.covers(
        slot
    ):

        row1.append(
            slot
        )

```

```

# -----
# SECOND ROW
# -----

for x in x_positions:

    slot = slot_geometry(
        x,
        row2_y,
        angle
    )

    if polygon.covers(
        slot
    ):

        row2.append(
            slot
        )

return (
    row1,
    row2,
    aisle,
    aisle_ratio,
    module_depth
)

# =====
# COMPLETE LAYOUT
# =====

def generate_layout(
    polygon,
    angle,
    y_offset
):

    (
        stall_depth,
        frontage,
        aisle_width,
        module_depth
    ) = module_dimensions(
        angle
    )

    minx, miny, maxx, maxy = (
        polygon.bounds
    )

    all_slots = []
    all_aisles = []

    # -----
    # Repeat double-sided modules
    # -----

    y = (
        miny
        +
        y_offset
    )

    while (
        y + module_depth
        <= maxy + 1e-9

```

```

):
    (
        row1,
        row2,
        aisle,
        aisle_ratio,
    ) = build_bay(
        polygon,
        y,
        angle
    )

    # -----
    # Accept bay only when aisle is mostly usable
    # -----

    if (
        aisle_ratio >= 0.80
        and
        len(row1) > 0
        and
        len(row2) > 0
    ):

        all_slots.extend(
            row1
        )

        all_slots.extend(
            row2
        )

        all_aisles.append(
            aisle
        )

    y += module_depth

    return (
        all_slots,
        all_aisles
    )

# =====
# SEARCH BEST OFFSET FOR EACH ANGLE
# =====

def optimize_angle(
    polygon,
    angle
):
    (
        stall_depth,
        frontage,
        aisle_width,
        module_depth
    ) = module_dimensions(
        angle
    )

    best_slots = []
    best_aisles = []
    best_offset = 0

    # Search all possible phase offsets.

```

```

#
# This avoids depending on an arbitrary
# first-row position.

offsets = np.linspace(
    0,
    module_depth,
    31
)

for offset in offsets:

    slots, aisles = (
        generate_layout(
            polygon,
            angle,
            offset
        )
    )

    if len(slots) > len(
        best_slots
    ):

        best_slots = slots
        best_aisles = aisles
        best_offset = offset

return {
    "angle": angle,
    "slots": best_slots,
    "aisles": best_aisles,
    "offset": best_offset,
    "aisle_width": aisle_width,
    "module_depth": module_depth
}

# =====
# RUN OPTIMIZATION
# =====

print("\n=====")
print("RUNNING ANGLE OPTIMIZATION")
print("=====")

results = []

for angle in ANGLES:

    result = optimize_angle(
        LOCAL_POLYGON,
        float(angle)
    )

    results.append(
        result
    )

    print(
        f"{angle:5.1f}°"
        f" → "
        f"{len(result['slots']):3d} slots"
    )

# =====
# BEST RESULT
# =====

```

```

best = max(
    results,
    key=lambda r: len(
        r["slots"]
    )
)

# =====
# SANITY CHECK
# =====

ECS_AREA = 23.0

area_based_upper_bound = (
    PARKING_AREA_M2
    /
    ECS_AREA
)

print("\n=====")
print("FINAL OPTIMIZATION")
print("=====")

print(
    f"Best angle      : "
    f"{best['angle']:.1f}°"
)

print(
    f"Best slot count  : "
    f"{len(best['slots'])}"
)

print(
    f"Aisle width      : "
    f"{best['aisle_width']:.2f} m"
)

print(
    f"Module depth      : "
    f"{best['module_depth']:.2f} m"
)

print(
    f"Plot area         : "
    f"{PARKING_AREA_M2:.2f} m²"
)

print(
    f"23 m²/ECS check   : "
    f"{area_based_upper_bound:.1f} ECS"
)

# =====
# TOP 10 CONFIGURATIONS
# =====

print("\n=====")
print("TOP CONFIGURATIONS")
print("=====")

top_results = sorted(
    results,
    key=lambda r: len(
        r["slots"]
    ),
    reverse=True
)

```

```

for r in top_results[:10]:

    print(
        f"{r['angle']:5.1f}°"
        f" → "
        f"{len(r['slots']):3d} slots"
    )

# =====
# VISUALIZE
# =====

fig, ax = plt.subplots(
    figsize=(15, 10)
)

# Boundary
bx, by = (
    LOCAL_POLYGON
    .exterior
    .xy
)

ax.plot(
    bx,
    by,
    linewidth=3
)

# Aisles
for aisle in best[
    "aisles"
]:

    ax.fill(
        *aisle.exterior.xy,
        alpha=0.12
    )

    ax.plot(
        *aisle.exterior.xy,
        linestyle="--",
        linewidth=1.5
    )

# Slots
for i, slot in enumerate(
    best["slots"],
    start=1
):

    sx, sy = (
        slot.exterior.xy
    )

    ax.fill(
        sx,
        sy,
        alpha=0.35
    )

    ax.plot(
        sx,
        sy,
        linewidth=0.8
    )

    c = slot.centroid

```

```
ax.text(
    c.x,
    c.y,
    str(i),
    fontsize=6,
    ha="center",
    va="center"
)

ax.set_title(
    "RESEARCH-BACKED PARKING OPTIMIZATION\n"
    f"{len(best['slots'])} slots @ "
    f"{best['angle']:.1f}°",
    fontsize=18,
    fontweight="bold"
)

ax.set_xlabel(
    "X (metres)"
)

ax.set_ylabel(
    "Y (metres)"
)

ax.set_aspect(
    "equal"
)

ax.grid(
    alpha=0.2
)

plt.show()
```


0.0° →	39 slots
1.0° →	38 slots
2.0° →	38 slots
3.0° →	37 slots
4.0° →	36 slots
5.0° →	35 slots
6.0° →	34 slots
7.0° →	34 slots
8.0° →	34 slots
9.0° →	25 slots
10.0° →	24 slots
11.0° →	24 slots
12.0° →	24 slots
13.0° →	24 slots
14.0° →	24 slots
15.0° →	24 slots
16.0° →	24 slots
17.0° →	23 slots
18.0° →	23 slots
19.0° →	23 slots
20.0° →	23 slots
21.0° →	23 slots
22.0° →	23 slots
23.0° →	23 slots
24.0° →	23 slots
25.0° →	23 slots
26.0° →	23 slots
27.0° →	23 slots
28.0° →	23 slots
29.0° →	23 slots
30.0° →	23 slots
31.0° →	23 slots